



Applied Data Engineering · Spring 2026

Polars DataFrame Library

A Lightning-Fast, Rust-Powered Alternative to Pandas for Production Data Engineering

COURSE

YZV 322E Applied Data Engineering

PRESENTED BY

Burak Emre Polat · 150230328

DATE

April 2026

What Is Polars?

Rust-native DataFrame library — compiled, not interpreted

Created by [Ritchie Vink](#), first released **2020**

License: **MIT** — fully open source

Built on [Apache Arrow](#) columnar memory format

Bindings: **Python** (primary), Rust, Node.js, R

Current version: **1.40** — reached 1.0 in mid-2024

Core Design Philosophy

Multi-threaded by default. Every query is compiled to a parallel execution plan. No configuration needed — all cores, all the time.

Key Insight

Polars is to Pandas what Rust is to Python — same job, modern foundation.

The Problem with Pandas in Production

Single-threaded — uses 1 of your 8 cores

Eager evaluation — every operation materialises a new DataFrame

NumPy memory model — not great for strings, dates, or nulls

No streaming — everything must fit in RAM

3 min

Time to read a 5 GB CSV in Pandas (Airflow DAG)

50×

DAGs running daily — that latency compounds fast

Pandas is excellent for exploration — but production pipelines need something built for modern hardware.

SOLUTION

How Polars Solves It

FEATURE	PANDAS	POLARS	DASK
Language	Python / C	Rust	Python
Parallelism	✗ No	✓ Yes (default)	✓ Yes
Lazy Evaluation	✗ No	✓ Yes	✓ Yes
Memory Model	NumPy	Apache Arrow	Pandas
Larger-than-RAM	✗ No	✓ Yes (streaming)	✓ Yes
API Style	Imperative	Expressions	Pandas-like

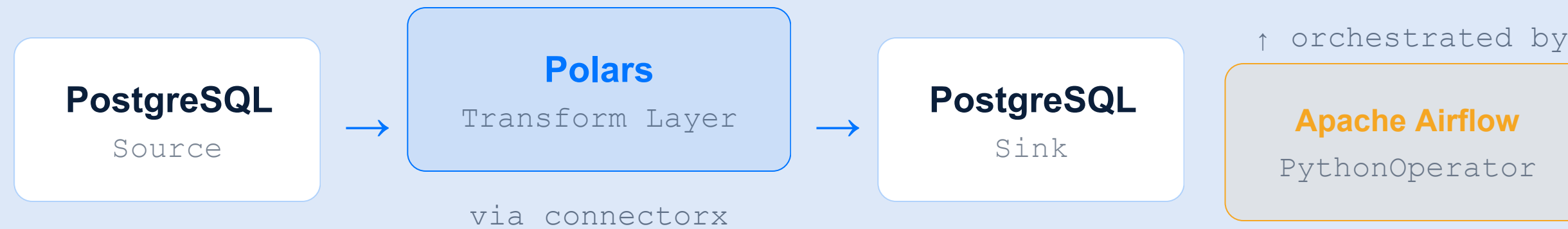
Lazy + Optimizer

Rewrites your query: predicate pushdown, projection pushdown, CSE elimination

vs Dask

Faster on a single machine, cleaner API, no cluster required

Polars in the Course Stack



PostgreSQL I/O

Reads and writes via **connectorx** — Rust-to-Rust, parallel query execution

Airflow Integration

Drop-in replacement for Pandas inside **PythonOperator** tasks

Docker-Ready

Outputs Parquet for NiFi/Elasticsearch downstream feeds

Setup with Docker

```
# docker-compose.yml - four services services :

postgres:          # data store
  image: postgres:16

app:               # ETL scripts
  build: .

jupyter:          # feature notebook
  build: .

  ports: ["8888:8888" ]

pgadmin:          # visual DB inspection
  image: dpage/pgadmin4:latest
  ports: ["5050:80"]
```

Reproduce everything in 3 commands

```
$ docker compose up -d --build

$ docker compose exec app \
  python data/load_taxi_data.py

$ docker compose exec app \
  python src/etl_pipeline.py
```

Services

postgres:16	app ETL	jupyter	pgadmin
Data store	runner	localhost:8888	localhost:5050

Feature Walkthrough

notebooks/polars_showcase.ipynb → localhost:8888

1 Postgres I/O via connectorx

Run cells — point at the **6–8× speedup** over Pandas. connectorx writes parallel directly to Arrow memory.

2 Eager API

Run the chained group-by — briefly show the result. Clean, readable expression syntax.

★ Lazy API + Query Optimizer — the wow moment

Pause on the **optimized=True** plan. Show the optimizer at work: **PROJECT 4/19 COLUMNS** — only 4 of 19 columns actually read · **SELECTION:** — all three filters merged and pushed to scan

4 Benchmark Chart

Run the bar-chart cell. Narrate the speedups. (Section 4 streaming: mention briefly, point to repo.)

End-to-End ETL Pipeline

```
docker compose exec app python src/05_etl_pipeline.py
```

EXTRACT

Reads **3M rows** from PostgreSQL via connectorx — parallel Arrow reads

~4s

TRANSFORM

Cleans, enriches, aggregates using the **lazy API** — optimizer plan logged

~1s

LOAD

Writes ~47K aggregated rows to `taxi_hourly_summary`

—

VERIFY

Row count, time range, top 5 busiest buckets

—

This code lives inside an Airflow **PythonOperator** task.

Same code on a 100× bigger dataset — just add the streaming engine flag.

Then → **pgAdmin at localhost:5050**

1

Postgres server already in the tree
Pre-configured — no manual setup needed

2

Drill into taxi ▶ Schemas ▶ public ▶ Tables
Point out `taxi_trips` (raw) and `taxi_hourly_summary` (written by Polars)

3

View/Edit Data → First 100 Rows
Right-click
`taxi_hourly_summary`

4

Query Tool — ad-hoc SQL
Closes the ETL loop: Postgres · Docker · pgAdmin

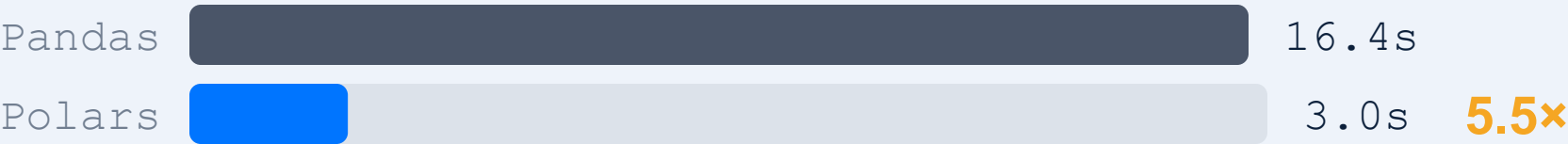
```
SELECT pu_location_id, SUM(trip_count) AS total_trips
FROM taxi_hourly_summary GROUP BY pu_location_id ORDER
BY total_trips DESC LIMIT 10;
```


RESULTS

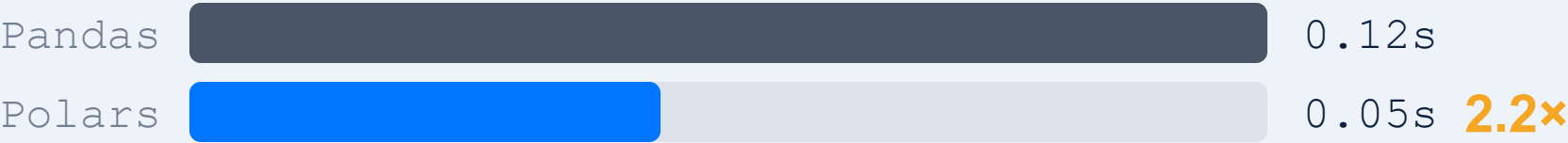
Benchmark Results & Honest Limitations

Operation Speedups (vs Pandas)

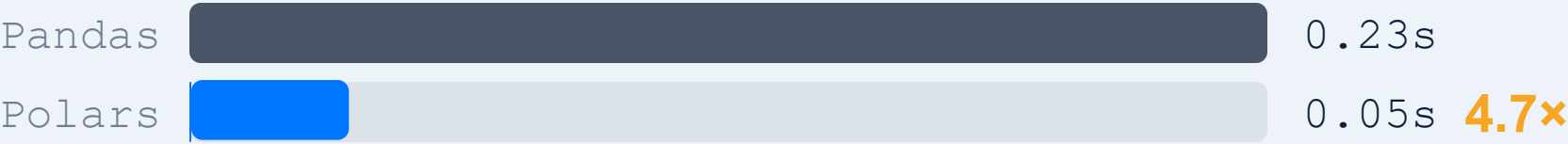
Read from PostgreSQL



Group-by + Aggregate



Aggregate + Join



Honest Limitations

Smaller ecosystem than Pandas — fewer Stack Overflow answers and third-party integrations

API still evolving — some breaking changes between 0.x and 1.x

Not a drop-in replacement — expression syntax requires rewriting existing code

For tiny datasets (<100K rows), Pandas may be slightly faster due to lower setup overhead

SOURCES

References

Polars Documentation — docs.pola.rs

Polars GitHub — github.com/pola-rs/polars

Apache Arrow — arrow.apache.org

connectorx — github.com/sfu-db/connector-x

Ritchie Vink's blog — "I wrote one of the fastest DataFrame libraries" (2021)

NYC TLC Trip Record Data — nyc.gov/tlc



Repository

`github.com/itu-itis23-polatb23/YZV322E-Tool-Presentation`

All notebooks, ETL scripts, Docker setup are available in the repo.

Dataset

NYC Yellow Taxi Trip Records — used to demonstrate real-world 3M-row data processing across all pipeline stages.

Thank You

YZV 322E Applied Data Engineering · Spring 2026